guide

HelixCode Troubleshooting Guide

This comprehensive guide provides detailed troubleshooting information for common HelixCode issues. Each section includes specific error messages, step-by-step diagnosis procedures, solutions, and prevention tips.

Table of Contents

---

1. Server Startup Issues

1.1 Common Errors and Solutions

Error: Configuration File Not Found

Error Message:

```
Error: failed to load configuration: open config/config.yaml: no such file
Fatal: configuration loading failed
```

Step-by-Step Diagnosis: 1. Verify you are in the correct directory (`helix_code/` subdirectory) 2. Check if config file exists: `ls -la config/config.yaml` 3. Verify file permissions: `stat config/config.yaml`

Solution:

```
# Ensure you're in the HelixCode directory
cd HelixCode

# Copy example configuration if available
cp config/config.yaml.example config/config.yaml

# Or create minimal configuration
cat > config/config.yaml << 'EOF'
server:
  address: "0.0.0.0"
```

```
  port: 8080
  read_timeout: 30
  write_timeout: 30

database:
  host: ""
  enabled: false

auth:
  jwt_secret: "${HELIX_AUTH_JWT_SECRET}"
  token_expiry: 86400

logging:
  level: "info"
  format: "text"
EOF
```

Prevention Tips: - Always verify working directory before running commands - Keep a backup of working configuration - Use environment-specific config files (e.g., `config/config.dev.yaml`)

Error: Invalid YAML Syntax

Error Message:

```
Error: failed to parse configuration: yaml: line 45: mapping values are no
Error: while parsing a block mapping at line 12, column 1
```

Step-by-Step Diagnosis: 1. Identify the problematic line from the error message 2. Check for common YAML issues: - Mixed tabs and spaces - Missing colons after keys - Incorrect indentation - Unquoted special characters

Solution:

```
# Validate YAML syntax
yamllint config/config.yaml

# Or use Python to check
python3 -c "import yaml; yaml.safe_load(open('config/
        config.yaml'))"

# Validate with HelixCode
./bin/helixcode --validate-config
```

Prevention Tips: - Use a YAML-aware editor with syntax highlighting - Always use spaces (not tabs) for indentation - Quote strings containing special characters (`:`, `#`, `{`, `}`, `[`, `]`) - Validate config changes before deploying

---

1.2 Port Conflicts

Error Message:

```
Error: listen tcp :8080: bind: address already in use
Fatal: failed to start HTTP server: address already in use
```

Step-by-Step Diagnosis: 1. Identify what process is using the port 2. Determine if it's another HelixCode instance or different service 3. Decide whether to terminate the process or change ports

Solution:

```
# Find process using port 8080
lsof -i :8080
# or
ss -tlnp | grep 8080
# or
netstat -tlnp | grep 8080

# Option 1: Kill the conflicting process
kill -9 <PID>

# Option 2: Change HelixCode port in config
# Edit config/config.yaml:
# server:
#   port: 8081

# Option 3: Use environment variable
HELIX_SERVER_PORT=8081 ./bin/helixcode
```

Common Port Conflicts: | Port | Common Service | |——|————-| | 8080 | Default HTTP, Jenkins, other apps | | 5432 | PostgreSQL | | 6379 | Redis | | 11434 | Ollama |

Prevention Tips: - Document which ports your environment uses - Use unique ports for development vs production - Implement proper shutdown procedures to release ports - Add health checks before starting services

---

1.3 Missing Dependencies

Error Message:

```
Error: cannot find package "github.com/gin-gonic/gin" in any of:
Error: exec: "chromium": executable file not found in $PATH
```

Step-by-Step Diagnosis: 1. Check if Go modules are downloaded 2. Verify external dependencies are installed 3. Check PATH environment variable

Solution:

```
# Download Go dependencies
cd HelixCode
go mod download
go mod tidy

# Or use make
make setup-deps

# Install system dependencies (Ubuntu/Debian)
sudo apt-get install -y chromium-browser

# Install system dependencies (RHEL/CentOS)
sudo dnf install -y chromium

# Verify Go installation
go version
which go
```

Required System Dependencies: - Go 1.26.0 or later - PostgreSQL client libraries (optional) - Redis client (optional) - Chromium/Chrome (for browser automation features) - Git (for development)

Prevention Tips: - Document all system dependencies in README - Use Docker for consistent environments - Run `go mod download` after cloning - Set up CI/CD to catch missing dependencies early

---

2. Database Connection Problems

2.1 Connection Refused

Error Message:

```
Error: failed to create connection pool: dial tcp 127.0.0.1:5432: connect:
Error: failed to ping database: connection refused
FATAL: could not connect to server: Connection refused
```

Step-by-Step Diagnosis: 1. Verify PostgreSQL service is running 2. Check if PostgreSQL is listening on the expected port 3. Verify network connectivity 4. Check firewall rules

Solution:

```
# Check PostgreSQL service status
systemctl status postgresql
# or for Docker
docker ps | grep postgres

# Start PostgreSQL if stopped
sudo systemctl start postgresql
# or
docker start helixcode-postgres
```

```
# Verify PostgreSQL is listening
ss -tlnp | grep 5432

# Test connection manually
psql -h localhost -p 5432 -U helix -d helixcode_prod

# Check PostgreSQL logs
sudo tail -f /var/log/postgresql/postgresql-*.log
# or Docker
docker logs helixcode-postgres

# If using Docker, ensure container is on correct network
docker network inspect helixcode_network
```

Prevention Tips: - Configure database service to start on boot - Implement connection retry logic in application - Use health checks in Docker Compose - Monitor database availability with alerts

---

2.2 Authentication Failures

Error Message:

```
Error: password authentication failed for user "helix"
FATAL: role "helix" does not exist
Error: FATAL: database "helixcode_prod" does not exist
pq: password authentication failed for user "helix"
```

Step-by-Step Diagnosis: 1. Verify database user exists 2. Check password is correct 3. Verify database exists 4. Check pg_hba.conf authentication settings

Solution:

```
# Check if user exists (as postgres superuser)
sudo -u postgres psql -c "SELECT usename FROM pg_user;"

# Create user if missing
sudo -u postgres psql -c "CREATE USER helix WITH PASSWORD
        'your_password';"

# Check if database exists
sudo -u postgres psql -c "SELECT datname FROM pg_database;"

# Create database if missing
sudo -u postgres createdb -O helix helixcode_prod

# Grant privileges
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE
        helixcode_prod TO helix;"

# Verify environment variable is set
```

```
echo $HELIX_DATABASE_PASSWORD

# Set password via environment
export HELIX_DATABASE_PASSWORD='your_secure_password'

# Check pg_hba.conf for authentication method
sudo cat /etc/postgresql/*/main/pg_hba.conf | grep -v "^#" |
        grep -v "^$"
```

Prevention Tips: - Use environment variables for credentials (never commit passwords) - Document required database setup steps - Use database migration scripts - Implement connection validation on startup

---

2.3 Schema Issues

Error Message:

```
Error: failed to check schema existence: relation "users" does not exist
Error: failed to initialize schema: permission denied for schema public
ERROR: column "display_name" of relation "users" does not exist
```

Step-by-Step Diagnosis: 1. Check if tables exist in database 2. Verify schema permissions 3. Check for pending migrations 4. Verify column definitions match code expectations

Solution:

```
# Connect to database and check tables
psql -h localhost -U helix -d helixcode_prod

# List all tables
\dt

# Check specific table schema
\d users

# Grant schema permissions
GRANT ALL ON SCHEMA public TO helix;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO helix;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO helix;

# Run schema initialization manually
# Option 1: Let application initialize
./bin/helixcode --init-schema

# Option 2: Run SQL script directly
psql -U helix -d helixcode_prod -f postgres-init.sql

# Reset database if needed (CAUTION: data loss)
dropdb helixcode_prod
createdb -O helix helixcode_prod
```

Prevention Tips: - Use database migrations (not auto-schema creation in production) - Keep schema versioned in source control - Test migrations in staging before production - Implement database backup before schema changes

---

3. LLM Provider Failures

3.1 API Key Issues

Error Message:

```
Error: authentication failed: invalid API key
Error: 401 Unauthorized: Incorrect API key provided
Error: API key not found in environment: OPENAI_API_KEY
Error: failed to authenticate with Anthropic: invalid x-api-key
```

Step-by-Step Diagnosis: 1. Verify API key environment variable is set 2. Check API key format and validity 3. Test API key directly with provider 4. Check for rate limiting or account issues

Solution:

```
# Check if API keys are set
echo $OPENAI_API_KEY | head -c 10
echo $ANTHROPIC_API_KEY | head -c 10
echo $GEMINI_API_KEY | head -c 10

# Set API keys
export OPENAI_API_KEY='sk-...'
export ANTHROPIC_API_KEY='sk-ant-...'
export GEMINI_API_KEY='AI...'

# Test OpenAI key
curl -H "Authorization: Bearer $OPENAI_API_KEY" \
     https://api.openai.com/v1/models \
     -s | head -20

# Test Anthropic key
curl -H "x-api-key: $ANTHROPIC_API_KEY" \
     -H "anthropic-version: 2023-06-01" \
     https://api.anthropic.com/v1/models \
     -s | head -20

# Check key in configuration
grep -r "api_key" config/config.yaml

# Verify provider is enabled
./bin/helixcode llm status
```

API Key Formats: | Provider | Key Prefix | Example | |————-|————|————| | OpenAI | `sk-` | `sk-abc123...` | | Anthropic | `sk-ant-` | `sk-ant-abc123...` | | Gemini | `AI` | `AIzaSy...` | | xAI | varies | Check xAI documentation |

Prevention Tips: - Store API keys in secure secrets manager - Use different keys for development/production - Monitor API usage to detect key compromise - Set up alerts for authentication failures - Rotate keys regularly

---

3.2 Model Not Available

Error Message:

```
Error: model not found: gpt-5-turbo
Error: The model 'claude-4-opus' does not exist
Error: no models available from provider
Error: invalid model name: gemini-pro-ultra
```

Step-by-Step Diagnosis: 1. List available models from provider 2. Check model name spelling and format 3. Verify account has access to requested model 4. Check if model is deprecated or renamed

Solution:

```
# List available models
./bin/helixcode llm models

# Check OpenAI available models
curl -H "Authorization: Bearer $OPENAI_API_KEY" \
     https://api.openai.com/v1/models | jq '.data[].id'

# Check Anthropic models (from documentation)
# claude-3-opus-20240229, claude-3-sonnet-20240229, claude-3-
        haiku-20240307

# Check Ollama local models
ollama list

# Update model configuration
# Edit config/config.yaml
llm:
  providers:
    openai:
      enabled: true
      parameters:
        default_model: "gpt-4o"  # Use correct model name
```

Common Model Names: | Provider | Available Models | |————-|————————| | OpenAI | gpt-4o, gpt-4-turbo, gpt-3.5-turbo | | Anthropic | claude-3-opus-20240229, claude-3-sonnet-20240229, claude-3-haiku-20240307 | | Ollama | llama2, mistral, codellama, mixtral | | Gemini | gemini-pro, gemini-pro-vision |

Prevention Tips: - Use model aliases for abstraction - Document supported models in your deployment - Subscribe to provider changelogs for deprecation notices - Test model availability before deploying config changes

---

3.3 Rate Limiting

Error Message:

```
Error: rate limit exceeded: too many requests
Error: 429 Too Many Requests
Error: You exceeded your current quota
Error: Rate limit reached for gpt-4 in organization
```

Step-by-Step Diagnosis: 1. Check current API usage 2. Identify which operations are consuming quota 3. Review rate limit headers from API responses 4. Check organization/account limits

Solution:

```
# Check rate limit headers in response
curl -v -H "Authorization: Bearer $OPENAI_API_KEY" \
     https://api.openai.com/v1/models 2>&1 | grep -i "x-
         ratelimit"

# Configure retry with backoff
# Edit config/config.yaml
llm:
  max_retries: 3
  retry_delay: "2s"
  retry_backoff_multiplier: 2

# Configure fallback providers
llm:
  selection:
    strategy: "availability"
    fallback_enabled: true

# Monitor usage
./bin/helixcode llm usage

# Implement request queuing for batch operations
./bin/helixcode llm queue --enable --max-concurrent 5
```

Rate Limit Strategies: 1. **Implement exponential backoff** - Wait longer between retries 2. **Use fallback providers** - Switch to backup provider when limited 3. **Cache responses** - Avoid repeated identical requests 4. **Batch requests** - Combine multiple small requests 5. **Upgrade tier** - Contact provider for higher limits

Prevention Tips: - Monitor API usage regularly - Set up usage alerts before hitting limits - Implement request caching where appropriate - Design systems to gracefully degrade under rate limits - Use different API keys for different environments

---

4. Worker Registration Issues

4.1 SSH Connection Failures

Error Message:

```
Error: failed to connect to worker: dial tcp 192.168.1.100:22: connect: co
Error: ssh: unable to authenticate
Error: ssh: handshake failed: ssh: unable to authenticate
Error: failed to add worker: connection timed out
```

Step-by-Step Diagnosis: 1. Verify network connectivity to worker host 2. Check SSH service is running on worker 3. Verify SSH port is correct 4. Test SSH connection manually

Solution:

```
# Test basic network connectivity
ping -c 3 worker-host

# Check if SSH port is open
nc -zv worker-host 22
# or
nmap -p 22 worker-host

# Test SSH connection manually
ssh -v user@worker-host

# Check SSH service on worker
ssh user@worker-host "systemctl status sshd"

# Verify SSH configuration on worker
ssh user@worker-host "cat /etc/ssh/sshd_config | grep -E '^(Port|
        PasswordAuthentication|PubkeyAuthentication)'"

# Check firewall on worker
ssh user@worker-host "sudo iptables -L | grep ssh"
# or
ssh user@worker-host "sudo ufw status"
```

Prevention Tips: - Document network requirements for workers - Use consistent SSH port across infrastructure - Implement network health monitoring - Set up SSH keepalive to prevent timeouts

---

4.2 Key Authentication Problems

Error Message:

```
Error: ssh: unable to authenticate, attempted methods [none publickey]
Error: permission denied (publickey)
```

```
Error: failed to read private key: no such file or directory
Error: ssh: cannot decode encrypted private key
```

Step-by-Step Diagnosis: 1. Verify private key file exists and is readable 2. Check key file permissions (should be 600) 3. Verify public key is in worker's authorized_keys 4. Check key format compatibility

Solution:

```
# Check key file exists
ls -la ~/.ssh/id_rsa
ls -la ~/.ssh/id_ed25519

# Fix key permissions
chmod 600 ~/.ssh/id_rsa
chmod 700 ~/.ssh
chmod 644 ~/.ssh/id_rsa.pub

# Test key authentication
ssh -i ~/.ssh/id_rsa -v user@worker-host

# Copy public key to worker
ssh-copy-id -i ~/.ssh/id_rsa.pub user@worker-host

# Manually add public key to worker
# On worker host:
cat >> ~/.ssh/authorized_keys << 'EOF'
ssh-rsa AAAAB3... your-key-here
EOF
chmod 600 ~/.ssh/authorized_keys

# Generate new key pair if needed
ssh-keygen -t ed25519 -C "helixcode@example.com" -f ~/.ssh/
        helixcode_key

# For encrypted keys, use ssh-agent
eval $(ssh-agent -s)
ssh-add ~/.ssh/id_rsa
```

SSH Key Configuration in HelixCode:

```
workers:
  ssh:
    private_key_path: "/home/helix/.ssh/id_rsa"
    # or embed key directly (not recommended)
    private_key: |
      -----BEGIN OPENSSH PRIVATE KEY-----
      ...
      -----END OPENSSH PRIVATE KEY-----
```

Prevention Tips: - Use Ed25519 keys for better security and compatibility - Store SSH keys securely (e.g., in secrets manager) - Implement key rotation procedures - Document key distribution process - Use SSH certificates for large deployments

---

4.3 Network Issues

Error Message:

```
Error: failed to connect: network is unreachable
Error: dial tcp: lookup worker-host: no such host
Error: i/o timeout during SSH handshake
Error: failed to establish worker connection: context deadline exceeded
```

Step-by-Step Diagnosis: 1. Verify DNS resolution 2. Check network routing 3. Test connectivity at various network layers 4. Check for VPN/firewall interference

Solution:

```
# Test DNS resolution
nslookup worker-host
dig worker-host
host worker-host

# Test network path
traceroute worker-host
# or
mtr worker-host

# Check local network interface
ip addr show
ip route show

# Test with IP address directly (bypass DNS)
ssh user@192.168.1.100

# Check for firewall rules blocking connection
sudo iptables -L -n | grep -E "(22|DROP|REJECT)"

# Verify VPN status if applicable
ip route | grep -E "(tun|tap)"

# Test from worker side
ssh user@worker-host "ss -tlnp | grep 22"

# Increase connection timeout
./bin/helixcode worker add \
  --host worker-host \
  --connect-timeout 60s
```

Network Troubleshooting Checklist: - [ ] DNS resolves correctly - [ ] Host is reachable (ping) - [ ] SSH port is open (nc/nmap) - [ ] No firewall blocking - [ ] VPN connected (if required) - [ ] Correct network interface used - [ ] No proxy interference

Prevention Tips: - Use IP addresses in config for critical workers - Implement connection pooling with keepalives - Set up network monitoring between master and workers - Document network topology and requirements - Use internal DNS for worker hostnames

---

5. Authentication Errors

5.1 JWT Issues

Error Message:

```
Error: token expired
Error: invalid token
Error: token signature is invalid
Error: JWT secret not configured
Error: failed to parse JWT token: token contains an invalid number of segm
```

Step-by-Step Diagnosis: 1. Check JWT secret is configured 2. Verify token has not expired 3. Check token format and signature 4. Verify clock synchronization

Solution:

```
# Check JWT secret is set
echo $HELIX_AUTH_JWT_SECRET | wc -c
# Should be at least 32 characters

# Set JWT secret
export HELIX_AUTH_JWT_SECRET='your-secure-secret-at-least-32-
        chars-long'

# Generate secure secret
openssl rand -base64 32

# Verify token structure (has 3 parts separated by dots)
echo "your.token.here" | tr '.' '\n' | wc -l
# Should output 3

# Decode token payload (for debugging)
echo "your.token.here" | cut -d'.' -f2 | base64 -d 2>/dev/null |
        jq .

# Check token expiration
./bin/helixcode auth validate-token <token>

# Check system clock
date
timedatectl status
```

```
# Sync clock if needed
sudo timedatectl set-ntp true
```

JWT Configuration:

```
auth:
  jwt_secret: "${HELIX_AUTH_JWT_SECRET}"  # Min 32 characters
  token_expiry: 86400  # 24 hours in seconds
  refresh_enabled: true
  refresh_expiry: 604800  # 7 days
```

Prevention Tips: - Use strong, random JWT secrets (32+ bytes) - Implement token refresh mechanism - Keep clocks synchronized (NTP) - Log authentication failures for security monitoring - Set appropriate token expiration times

---

5.2 Session Problems

Error Message:

```
Error: session not found
Error: session expired
Error: failed to store session: WRONGTYPE Operation against a key holding
Error: invalid session token
```

Step-by-Step Diagnosis: 1. Check session storage (Redis) is available 2. Verify session hasn't expired 3. Check session data integrity 4. Verify session ID format

Solution:

```
# Check Redis connectivity
redis-cli ping

# List all sessions
redis-cli keys "session:*"

# Check specific session
redis-cli get "session:<session_id>"

# Check session TTL
redis-cli ttl "session:<session_id>"

# Clear specific session
redis-cli del "session:<session_id>"

# Clear all sessions (CAUTION: logs out all users)
redis-cli keys "session:*" | xargs redis-cli del

# Check session configuration
grep -A 5 "session" config/config.yaml
```

```
# Verify Redis is enabled
./bin/helixcode redis status
```

Session Configuration:

```
auth:
  session_expiry: 604800  # 7 days in seconds
  session_storage: "redis"  # or "memory"

redis:
  enabled: true
  host: "localhost"
  port: 6379
```

Prevention Tips: - Monitor Redis memory and performance - Implement session cleanup for expired sessions - Use consistent session ID format - Log session creation/invalidation for audit - Implement graceful session refresh

---

5.3 Permission Denied

Error Message:

```
Error: permission denied: insufficient privileges
Error: access denied for resource: /api/v1/admin/users
Error: role 'user' cannot perform action 'delete' on resource 'project'
Error: authorization failed: missing required scope
```

Step-by-Step Diagnosis: 1. Check user's assigned roles 2. Verify role permissions configuration 3. Check if resource requires specific permissions 4. Verify API endpoint authorization

Solution:

```
# Check user roles
./bin/helixcode user info <user_id>
./bin/helixcode user roles <user_id>

# Assign role to user
./bin/helixcode user assign-role <user_id> admin

# List available roles
./bin/helixcode roles list

# Check role permissions
./bin/helixcode roles show admin

# View API endpoint permissions
./bin/helixcode api permissions /api/v1/admin/users
```

```
# Debug authorization
./bin/helixcode auth debug --user <user_id> --resource
        <resource> --action <action>
```

Role Configuration:

```
auth:
  roles:
    admin:
      - "read:*"
      - "write:*"
      - "delete:*"
      - "admin:*"
    developer:
      - "read:projects"
      - "write:projects"
      - "read:tasks"
      - "write:tasks"
    viewer:
      - "read:projects"
      - "read:tasks"
```

Prevention Tips: - Document required permissions for each feature - Use principle of least privilege - Implement permission inheritance carefully - Log all authorization failures - Regular audit of role assignments

---

6. Memory Provider Issues

6.1 Provider-Specific Troubleshooting

Mem0 Provider

Error Message:

```
Error: operation not supported by Mem0 API
Error: failed to connect to Mem0: connection refused
Error: Mem0 API rate limit exceeded
```

Solution:

```
# Check Mem0 configuration
grep -A 10 "mem0" config/config.yaml

# Test Mem0 API connection
curl -H "Authorization: Bearer $MEM0_API_KEY" \
     https://api.mem0.ai/v1/health

# Verify API key
echo $MEM0_API_KEY | head -c 10
```

Zep Provider

Error Message:

```
Error: zep does not support manual index management
Error: zep cloud does not support direct backup/restore operations
Error: failed to create Zep session: invalid project ID
```

Solution:

```
# Check Zep configuration
grep -A 10 "zep" config/config.yaml

# Test Zep connection
curl -H "Authorization: Api-Key $ZEP_API_KEY" \
     $ZEP_API_URL/api/v2/sessions

# Verify environment variables
echo $ZEP_API_KEY
echo $ZEP_API_URL
```

Character.AI Provider

Error Message:

```
Error: Character.AI does not provide a public API
Error: operation running in simulation mode
Error: character not found
Error: conversation not found
```

Note: Character.AI provider operates in simulation mode as there is no public API.

6.2 Connection Problems

Error Message:

```
Error: memory provider not initialized
Error: failed to store memory: connection refused
Error: memory retrieval timeout
```

Step-by-Step Diagnosis: 1. Verify provider is configured and enabled 2. Check provider-specific API connectivity 3. Verify credentials and authentication 4. Check for rate limiting

Solution:

```
# Check memory provider configuration
grep -A 20 "memory:" config/config.yaml

# Test provider status
```

```
./bin/helixcode memory status

# List configured providers
./bin/helixcode memory providers

# Test specific provider
./bin/helixcode memory test --provider zep

# Check provider logs
tail -f /var/log/helixcode/memory.log
```

Memory Configuration:

```
memory:
  default_provider: "zep"
  providers:
    zep:
      enabled: true
      api_url: "${ZEP_API_URL}"
      api_key: "${ZEP_API_KEY}"
    mem0:
      enabled: false
      api_key: "${MEM0_API_KEY}"
```

Prevention Tips: - Monitor memory provider health - Implement fallback memory storage - Cache frequently accessed memories - Set appropriate timeouts for memory operations - Log memory operation failures for debugging

---

7. Build/Compilation Problems

7.1 Missing Dependencies

Error Message:

```
Error: cannot find package "github.com/gin-gonic/gin"
go: downloading error: module not found
Error: missing go.sum entry for module
```

Step-by-Step Diagnosis: 1. Check if Go modules are initialized 2. Verify go.mod and go.sum exist 3. Check network connectivity to module proxy 4. Verify module cache integrity

Solution:

```
# Ensure you're in HelixCode directory
cd HelixCode

# Download all dependencies
go mod download

# Tidy dependencies
```

```
go mod tidy

# Clear module cache (if corrupted)
go clean -modcache

# Verify modules
go mod verify

# Download specific package
go get github.com/gin-gonic/gin@latest

# Use make target
make setup-deps

# Check Go proxy settings
go env GOPROXY
# Reset to default if needed
go env -w GOPROXY=https://proxy.golang.org,direct
```

Prevention Tips: - Keep go.mod and go.sum in source control - Run `go mod tidy` before committing - Use `go mod vendor` for offline builds - Document build dependencies in README

7.2 Go Version Issues

Error Message:

```
Error: go: go.mod requires go >= 1.24
Error: build constraints exclude all Go files
Error: undefined: slices.Contains
Error: type parameter requires go1.18 or later
```

Step-by-Step Diagnosis: 1. Check current Go version 2. Verify go.mod requirements 3. Check for incompatible features used 4. Verify GOROOT and GOPATH

Solution:

```
# Check Go version
go version
# Required: go1.26.0 or later

# Check go.mod requirements
head -5 go.mod
# Should show: go 1.26.3

# Install correct Go version (Ubuntu/Debian)
sudo rm -rf /usr/local/go
wget https://go.dev/dl/go1.26.3.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.26.3.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin
```

```
# Using gvm (Go Version Manager)
gvm install go1.26.3
gvm use go1.26.3

# Using asdf
asdf install golang 1.24.9
asdf global golang 1.24.9

# Verify environment
go env GOROOT
go env GOPATH
```

Prevention Tips: - Document required Go version in README - Use Go toolchain directives in go.mod - Test builds with target Go version in CI - Consider using Docker for consistent build environment

---

7.3 CGO Problems

Error Message:

```
Error: exec: "gcc": executable file not found
Error: cgo: C compiler not found
Error: undefined reference to 'sqlite3_open'
Error: cannot find -lsqlite3
Error: #include <tree_sitter/parser.h> file not found
```

Step-by-Step Diagnosis: 1. Check if CGO is required for the build 2. Verify C compiler is installed 3. Check for required C libraries 4. Verify pkg-config can find libraries

Solution:

```
# Check CGO status
go env CGO_ENABLED

# Install C compiler (Ubuntu/Debian)
sudo apt-get install -y build-essential

# Install C compiler (RHEL/CentOS)
sudo dnf groupinstall -y "Development Tools"

# Install specific libraries (tree-sitter example)
# Ubuntu/Debian
sudo apt-get install -y libtree-sitter-dev

# Install pkg-config
sudo apt-get install -y pkg-config

# Build without CGO (if possible)
CGO_ENABLED=0 go build -o bin/helixcode ./cmd/server
```

```
# Check library paths
pkg-config --libs tree-sitter
ldconfig -p | grep tree-sitter

# Set library path
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/lib
export CGO_LDFLAGS="-L/usr/local/lib"
export CGO_CFLAGS="-I/usr/local/include"
```

CGO Dependencies for HelixCode: | Feature | Library | Package (Ubuntu) | |———|———|——————| | Tree-sitter | libtree-sitter | libtree-sitter-dev | | SQLite (optional) | libsqlite3 | libsqlite3-dev |

Prevention Tips: - Document CGO requirements clearly - Provide static builds when possible - Use Docker for reproducible builds - Consider pure Go alternatives for dependencies

---

8. Test Failures

8.1 Common Test Issues

Error Message:

```
Error: test timed out after 30s
Error: panic: runtime error: invalid memory address
Error: connection refused (test couldn't connect to dependency)
Error: expected X but got Y
```

Step-by-Step Diagnosis: 1. Identify which test is failing 2. Check test dependencies (database, Redis) 3. Verify test configuration 4. Check for race conditions

Solution:

```
# Run all tests with verbose output
cd HelixCode
go test -v ./...

# Run specific package tests
go test -v ./internal/auth

# Run single test
go test -v ./internal/auth -run TestSpecific

# Run with race detection
go test -race ./...

# Run with extended timeout
go test -timeout 120s ./...

# Check test coverage
```

```
go test -cover ./...

# Generate coverage report
go test -coverprofile=coverage.out ./...
go tool cover -html=coverage.out

# Use test configuration
HELIX_CONFIG_PATH=config/test-config.yaml go test ./...
```

Prevention Tips: - Keep tests independent and isolated - Use table-driven tests - Mock external dependencies - Set appropriate timeouts - Run tests in CI before merging

---

8.2 Environment Setup

Error Message:

```
Error: no such host (database connection)
Error: Redis not available
Error: test requires infrastructure (skipped)
Error: missing test configuration
```

Step-by-Step Diagnosis: 1. Check if test infrastructure is running 2. Verify test configuration file 3. Check environment variables for tests 4. Determine if tests require external services

Solution:

```
# Use test configuration
cp config/test-config.yaml config/config.yaml

# Start test infrastructure with Docker
docker-compose -f docker-compose.test.yml up -d

# Or use make target
make test-infra-up

# Wait for services to be ready
sleep 10

# Run tests
make test

# Or use full test suite
make test-full

# Run unit tests only (no external dependencies)
./run_tests.sh

# Run all tests including integration
./run_all_tests.sh
```

```
# Stop test infrastructure
make test-infra-down
docker-compose -f docker-compose.test.yml down

# Check test infrastructure status
make test-infra-status
```

Test Configuration (config/test-config.yaml):

```
server:
  port: 8081  # Different from production

database:
  host: "localhost"
  port: 5433  # Different from production
  dbname: "helixcode_test"

redis:
  enabled: true
  host: "localhost"
  port: 6380  # Different from production

auth:
  jwt_secret: "test-secret-for-testing-only"
```

Test Infrastructure Checklist: - [ ] PostgreSQL test instance running - [ ] Redis test instance running - [ ] Test configuration file in place - [ ] Environment variables set - [ ] Network accessible between test runner and services - [ ] Sufficient disk space for test data

Prevention Tips: - Document test environment requirements - Use Docker Compose for consistent test infrastructure - Separate unit tests from integration tests - Skip integration tests in quick feedback loops - Use mocks for external service unit tests

---

Quick Reference: Environment Variables

Variable	Purpose	Required
`HELIX_AUTH_JWT_SECRET`	JWT token signing	Yes
`HELIX_DATABASE_PASSWORD`	PostgreSQL password	If DB enabled
`HELIX_DATABASE_HOST`	PostgreSQL host	If DB enabled
`HELIX_REDIS_PASSWORD`	Redis password	If Redis enabled
`OPENAI_API_KEY`	OpenAI API access	If using OpenAI
`ANTHROPIC_API_KEY`	Anthropic API access	If using Anthropic
`GEMINI_API_KEY`	Google Gemini access	If using Gemini
`ZEP_API_KEY`	Zep memory provider	If using Zep
`ZEP_API_URL`	Zep API endpoint	If using Zep

Quick Reference: Common Commands

```
# Server
./bin/helixcode                      # Start server
./bin/helixcode --config path.yaml   # Start with specific config
./bin/helixcode --validate-config    # Validate configuration

# Build
make build                           # Build server
make clean                           # Clean build artifacts
make setup-deps                      # Download dependencies

# Test
make test                            # Run tests
make test-coverage                   # Run with coverage
go test -v ./internal/auth           # Test specific package

# Debug
./bin/helixcode health              # Check system health
./bin/helixcode llm status          # Check LLM providers
./bin/helixcode worker list         # List workers
./bin/helixcode diagnostics collect # Collect diagnostic info
```

Getting Help

If this guide doesn't resolve your issue:

1. **Check Logs**: `tail -f /var/log/helixcode/server.log`
2. **Enable Debug Logging**: Set `logging.level: "debug"` in config
3. **Collect Diagnostics**: `./bin/helixcode diagnostics collect`
4. **Review Documentation**: See `docs/` directory for detailed guides
5. **Search Issues**: Check GitHub issues for similar problems
6. **Contact Support**: Open a new issue with diagnostic information

When Reporting Issues, Include: - HelixCode version (`./bin/helixcode --version`) - Go version (`go version`) - Operating system (`uname -a`) - Full error message and stack trace - Relevant configuration (redact secrets) - Steps to reproduce

Sources verified

Sources verified 2026-05-29: https://go.dev/dl/ (go1.26.3 latest stable Go; 1.24 past support) ; project go.mod (root go 1.25.2, inner go 1.26) + CLAUDE.md §3.1 (PostgreSQL 15+).